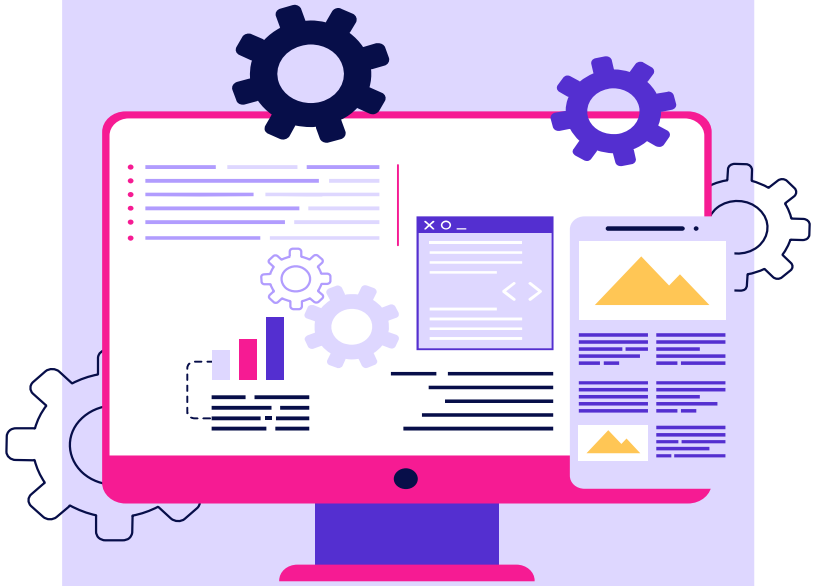
ED2 – Métodos de Ordenação

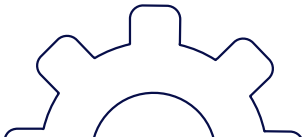
Profª Juliana Franciscani



Métodos de Ordenação Lineares

Usados para grandes volumes de dados:

- Counting Sort
- Bucket Sort
- Radix Sort



Counting Sort

- **Conhecido como ordenação por contagem:**
  - Algoritmo de ordenação para valores inteiros
  - Valores dentro de um determinado intervalo
  - A cada passo conta o número de ocorrências de um determinado valor no array

Funcionamento

- Usa um array auxiliar de tamanho igual ao maior valor a ser ordenado, K
- O array auxiliar é usado para contar quantas vezes cada valor ocorre
- Valor a ser ordenado é tratado como índice.
- Percorre o array auxiliar verificando quais valores existem e os coloca no array ordenado

Passo a passo

Sem Ordenar

5	6	10	1	3	1	7

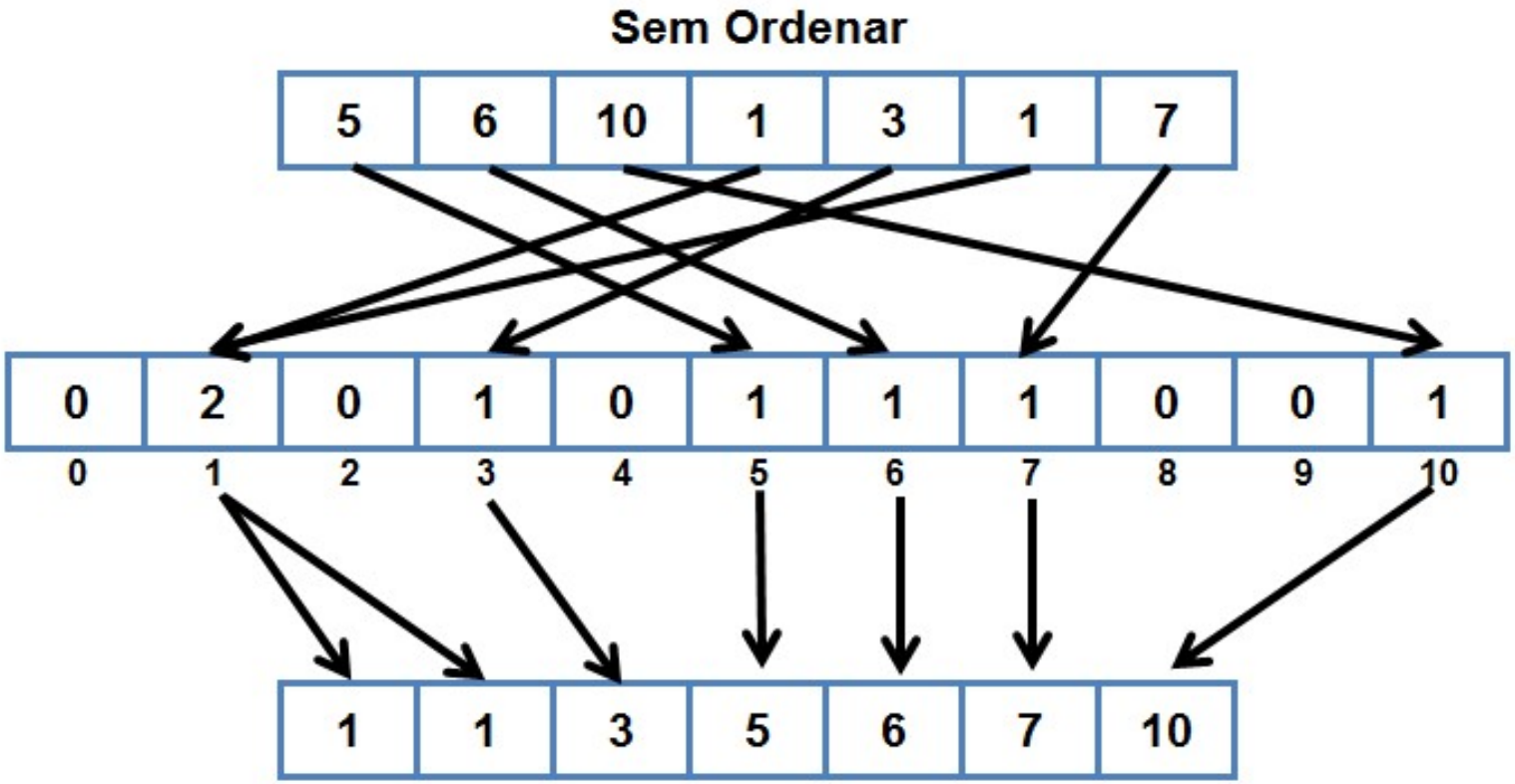
0	0	0	0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9	10

Sem Ordenar

5	6	10	1	3	1	7

0	2	0	1	0	1	1	1	0	0	1
0	1	2	3	4	5	6	7	8	9	10

Passo a passo



Código

```
void countingSort(int vet[], int n, int maior) {
    if (n <= 0) return;

    int i,j,x,tam = maior+1;
    int* vetCount = new int[tam];
    // Inicializar vetor de contagem com zero
    for (i = 0; i < tam; i++)
        vetCount[i] = 0;

    // Armazenar a frequência de cada elemento
    for (i = 0; i < n; i++)
        vetCount[vet[i]]++;
    // Percorrer o vetor de contagem de trás para frente
    for(i=n-1,j=tam-1;i>=0;j--)
        for(x=vetCount[j];x>0;x--){
            vet[i]=j;
            i--;
        }
    }
    // Limpar memória alocada dinamicamente
    delete[] vetCount;
}
```

Código comentado

```
void countingSort(int vet[], int n, int maior) {
    if (n <= 0) return;
    int i, j, x;
    int tam = maior + 1;
    int* vetCount = new int[tam];

    // Inicializar vetor de contagem com zero
    for (i = 0; i < tam; i++)
        vetCount[i] = 0;

    // Contar frequência dos elementos,
    //acessar vet (vetor original) e atualizar o vetCount
    for (i = 0; i < n; i++){
        pos = vet[i]; // o valor no vet é posição do vetcount
        vetCount[pos]++;
    }
    // Reconstruir vetor ordenado
    for (i = 0, j = 0; i < n; ) {
        //acessa o vetCount de trás para frente
        for (x = vetCount[j]; x > 0; x--) { //último elemento sempre presente
            vet[i] = j; // a posição do vetCount (se diferente 0), será um valor do vet
            i++;
        }
        j++;
    }
    delete[] vetCount;
}
```

Radix Sort

RadixSort é um algoritmo de ordenação rápido e estável que pode ser usado para ordenar itens que estão identificados por chaves únicas e que cada chave é uma cadeia de caracteres ou número.

Duas classificações podem ser realizadas com o radixsort,
LSD (Least Significant Bits) => inicia pela unidade

MSD (Most Significant Bits) => finaliza na unidade

Funcionamento

- Encontra o maior número do vetor para determinar quantas passagens do Counting Sort serão necessárias (baseado no número de dígitos).
- Para cada dígito (começando da unidade, depois dezena, centena, etc.), a função chama a countingSort para ordenar os números de acordo com esse dígito específico.

Passo a passo

Sem Ordenar

23	4	17	8	29	14	21

Faz o COUNTING para UNIDADES

	1		1	2			1	1	1
0	1	2	3	4	5	6	7	8	9

21	23	4	14	17	8	29
0	1	2	3	4	5	6

Faz o COUNTING para DEZENAS

2	2	3	0	2			0	0	0
0	1	2	3	4	5	6	7	8	9

4	8	14	17	21	23	29
0	1	2	3	4	5	6

```
void radixSort(int vet[], int n, int maior){
    int exp; //servirá para definir qual digito será analisado
    //exp =1 => verificará a unidade, exp =10=>será dezena, exp = 100 centena..
    for(exp=1; maior/exp>0; exp*=10)
        countingRadix(vet,n,exp); //não precisará passar o maior elemento,
                                  //vetCount será criado de tamanho 10 (base 10)
}
```

Passarei duas formas de implementação do countingRadix:

- 1ª) Usando o VetCount (vetor de frequência):
  - VetCount para verificar a presença de elementos com aquele dígito no vetor
  - Segue o raciocínio e metodologia do countingSort

- 2ª) Sem usar o VetCount:
  - Utiliza direto o valor do vetor original para inserir o elemento no vetor Auxiliar

```
void countingRadix(int vet[], int n, int exp){
    int i, dig, base =10;//10 por considerar a análise de números na base 10
    int *vetCount = new int[base];
    // inicializar o vetor de contagem
    for(i=0;i<10;i++)
        vetCount[i] = 0;
    // contar ocorrencia de elementos com cada digito (de acordo com exp)
    for(i=0;i<n;i++){
        dig = (vet[i]/exp)%10; //para encontrar o digito a ser analisado
        vetCount[dig]++;
    }
    int *aux = new int[n]; //cria vetor aux para armazenar os valores ordenados por digito
   //reposicionamento percorrendo o vetor original usando o vetCount
    int qt, pos = 0; // posição no vetor auxiliar
    for(dig=0; dig<base; dig++){  // percorre os dígitos
       qt = vetCount[dig]; // quantidade de ocorrencia no vetCount
       for(i=0; (i<n) && (qt>0); i++){ // percorre o vetor original
         if((vet[i]/exp)%10 == dig){
           aux[pos] = vet[i];
           pos++;
           qt--;
         }
       }
    }
    // copiar para vetor original
    for(int i=0;i<n;i++)
        vet[i] = aux[i];
    delete [] aux;
    delete [] vetCount;
}
```

Versão 1

```
void countingRadix(int vet[], int n, int exp){
    int i, dig, base =10;//10 por considerar a análise de números na base 10

    int *aux = new int[n]; //cria vetor aux para armazenar os valores ordenados por digito

  // reposicionamento percorrendo vetor original sem usar o vetCount
    int pos=0;
    for(dig=0; dig<base; dig++){
        for(i=0;i<n;i++){
            if((vet[i]/exp)%10 == dig){
                aux[pos] = vet[i];
                pos++;
            }
        }
    }
    // copiar para vetor original
    for(int i=0;i<n;i++)
        vet[i] = aux[i];
    delete [] aux;
}
```

Versão 2

Bucket Sort

- **Conhecido como ordenação com baldes:**
  - Algoritmo de ordenação para valores inteiros
  - Usa um conjunto de K baldes para separar os dados
  - A ordenação dos valores é feita por balde

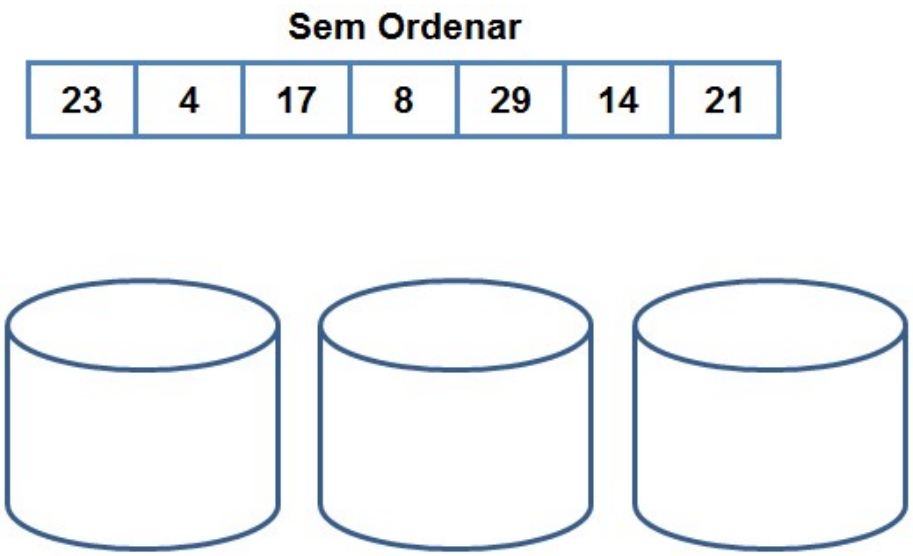
Funcionamento

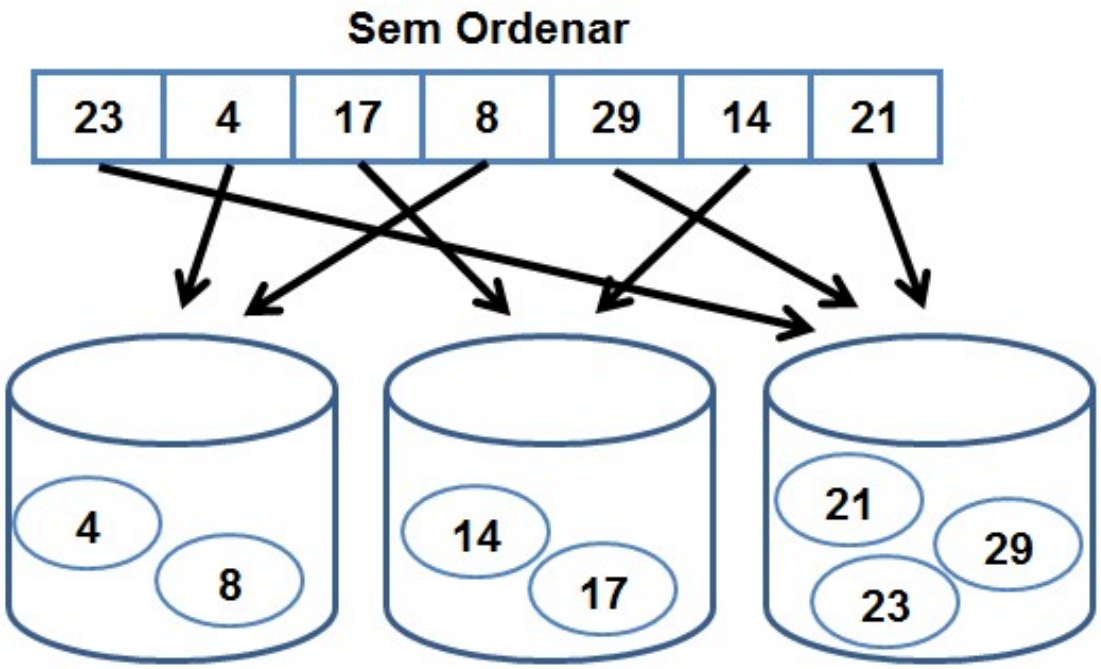
- Distribui os valores a serem ordenados em um conjunto de baldes.
  - Cada balde é um array auxiliar
  - Cada balde guarda uma faixa de valores
- Ordena os valores de cada balde.
  - Utiliza outro algoritmo de ordenação ou ele mesmo
- Percorre os baldes e coloca os valores de cada balde de volta no array ordenado

Etapas

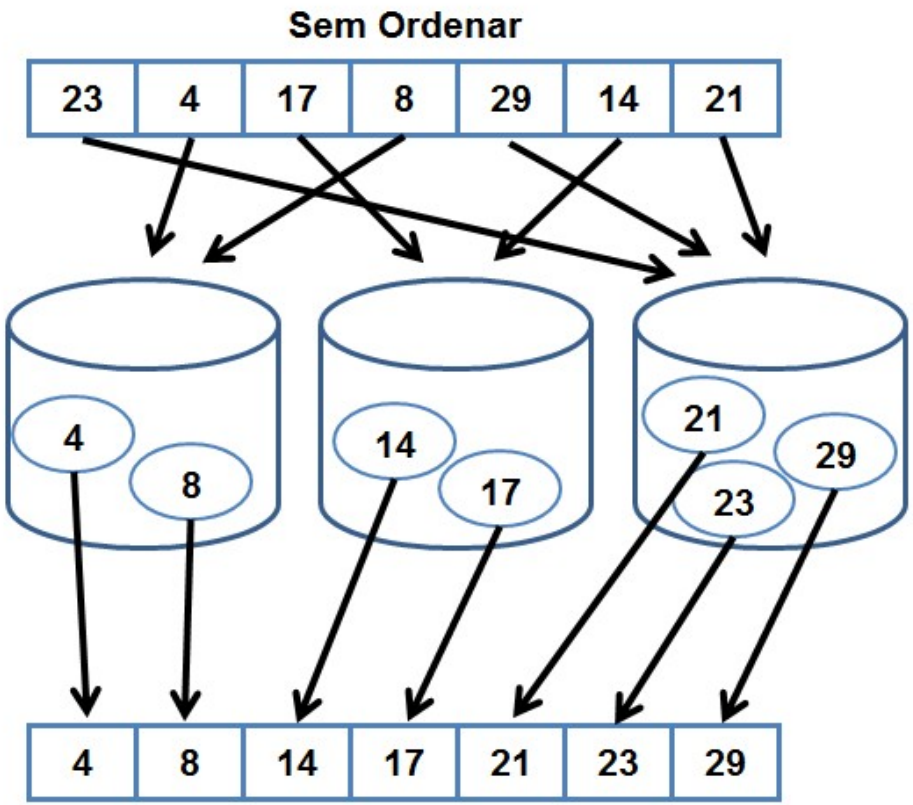
- Crie uma matriz vazia, ou uma lista de lista.
- Faça um loop pelo array original e coloque cada objeto em um “bucket”.
- Classifique cada um dos baldes não vazios
- Verifique os buckets em ordem e depois coloque todos os objetos de volta na matriz original.

Passo a passo





Passo a passo



Duas implementações distintas :

1ª Opção (sem o código, mas expliquei)

- Calculando o intervalo que cada balde terá
- Neste caso pode existir baldes sem elementos
- Não é muito eficiente pois deverá fazer várias verificações (de acordo com a quantidade de baldes)

2ª Opção (duas versões)

- Através de uma fórmula verifica em qual balde será inserido o elemento acessado no vetor original
- Mais eficiente pois não há necessidade de vários if...else, a inserção já é direta

Utilizaremos na implementação a matriz. Mas para ser mais eficiente o uso de lista é mais adequado.

Versão 1

```
void bucketSort(int vet[], int n, int maior){
    int i,j;
    int numBaldes = 5; // definido pelo programador
    // matriz de buckets
    int matrizBaldes[numBaldes][n];
    // vetor para controlar tamanho de cada bucket
    int qtElBalde[numBaldes] ={0} //vetor criado para armazenar a qtd de elementos de cada balde


    // distribuir elementos na matriz
    int indiceBalde; // criado para indicar qual balde o elemento será inserido
    int pos; //criado para indicar em qual posição o elemento será inserido no balde
    for(i=0;i<n;i++){
        indiceBalde = (vet[i] * numBaldes) / (maior + 1);  //cálculo de qual balde o número será inserido
        pos =qtElBalde[indiceBalde]; // pega quantidade de elementos atual no vetor de qtd do balde
        matriz[indiceBalde][pos] = vet[i];
        qtElBalde[indiceBalde]++; //atualiza a quantidade de elementos no balde
    }
```

```
    // ordenar os elementos em cada balde (linha)
    for(i=0;(i<numBaldes) && (qtElBalde [i]>0);i++) //  chama o insertion apenas para os baldes
        insertionSort(matriz[i], qtElBalde[i]); // passa a linha referente ao balde para o inser

    // juntar buckets no vetor original
    int posVet = 0;

    for(i=0;i<numBaldes;i++){
        for(j=0;j<qtElBalde[i];j++){
            vet[posVet] = matriz[i][j];
            posVet++;
        }
    }
}
```

Versão 2

```
// Função principal do Bucket Sort
void bucketSort(int *V, int n) {
    if (n <= 0) return;

    // Encontrando o valor mínimo e máximo no array
    int minVal = V[0], maxVal = V[0];
    for (int i = 1; i < n; i++) {
        if (V[i] < minVal) minVal = V[i];
        if (V[i] > maxVal) maxVal = V[i];
    }

    // Número de baldes buckets
    int numBaldes = 10;
    int intervalo = maxVal - minVal + 1;
    int tam = (intervalo + numBaldes - 1) / numBaldes;

    // Criando baldes, alocando um array para cada um
    int baldes[numBaldes][n];  // Matriz para armazenar
    int tamBaldes[numBaldes]={0};  // Índice para saber
```

```
int qtd;
// Distribuindo os elementos nos baldes
for (int i = 0; i < n; i++) {
    qtd = (V[i] - minVal) / tam;
    if (qtd >= numBaldes) qtd = numBaldes - 1;  // Para garantir q
    baldes[qtd][tamBaldes[qtd]++] = V[i];
}

// Ordenando cada balde usando o método de ordenação por inserção
for (int i = 0; i < numBaldes; i++) {
    if (tamBaldes[i] > 0) {
        insertionSort(baldes[i], tamBaldes[i]);
    }
}

// Recolocando os elementos no array original
int x = 0;
for (int i = 0; i < numBaldes; i++) {
    for (int j = 0; j < tamBaldes[i]; j++) {
        V[x] = baldes[i][j];
        x++;
    }
}
```

Exercícios

- A partir do vetor de tamanho 10 a seguir, faça a ordenação utilizando os métodos de ordenação lineares

- 10, 1, 7, 2, 8, 12, 9, 22, 18, 2
  - Counting
  - Radix
  - Bucket

Referências

EDELWEISS, N,; GALANTE, R.. Estruturas de dados. Porto Alegre: Bookman, 2009.

SZWARCFITER, J. L.; MARKENZON, L. Estruturas de dados e seus algoritmos. 3. ed. Rio de Janeiro: LTC, 2010.

ZIVIANI, N. Projeto de algoritmos: com implementações em Pascal e C. 3. ed. rev. e ampl. São Paulo: Cengage Learning, 2011.

Aulas e vídeo aulas do professor André Backes: https://www.facom.ufu.br/~backes/